REPUBLIC POLYTECHNIC
SCHOOL OF INFOCOMM

REPUBLIC POLYTECHNIC
Academy for Continuing Education

STREAMLIT-BASED-DIABETES-RISK-CLASSIFICATION-USING-MACHINE-LEARNING

Specialist Diploma in Applied Artificial Intelligence
Capstone Project 2021

IMPORTED LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler
from imblearn.combine import SMOTEENN
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import joblib
```

Imported Python libraries for data manipulation, visualization, preprocessing, resampling, model training, evaluation, and saving the trained model.

LOAD DATA

```
# load the data file
diabetes_df = pd.read_csv("data.csv")
# display rows
print(diabetes_df.head())

# display data info
print(diabetes_df.info())
```

Loaded the diabetes dataset from data.csv, displayed first few records, inspected the data information.

INSPECT AND DATA CLEANING

```
# check the number of duplicated rows
print(diabetes_df.duplicated().sum())

# remove duplicateed rows
diabetes_df.drop_duplicates(inplace = True)

# re-check the number of duplicated rows
print(diabetes_df.duplicated().sum())

# check the number of missing data in each col
print(diabetes_df.isnull().sum())
```

Checked for duplicate and missing values. Removed duplicated rows from the dataset and rechecked that no duplicates remained. Missing values were also checked before modelling.

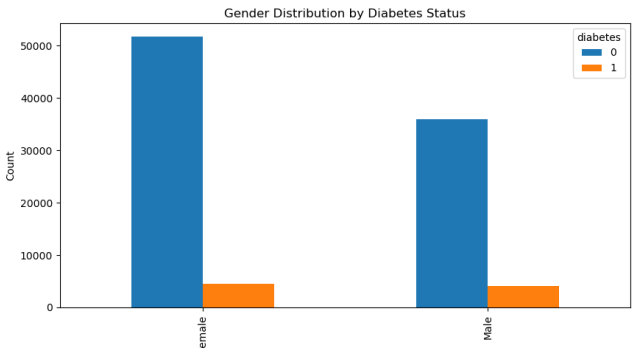
CATEGORICAL ENCODING

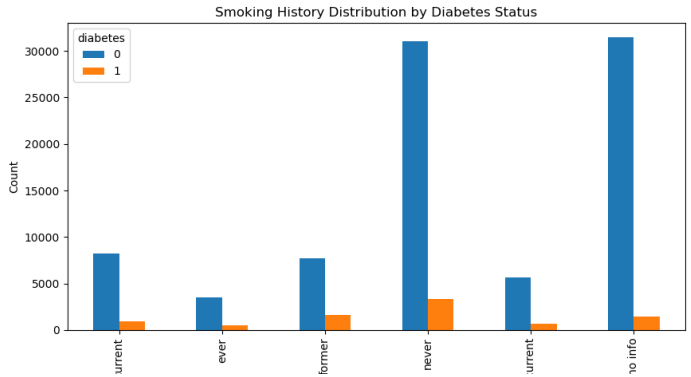
```python
# define custom function for the gender conversion
def gender_to_numeric(gender):
    if gender == "Female":
        return 0
    else:
        return 1

# use apply function to transform the categorical column into a numeric column
diabetes_df['gender'] = diabetes_df['gender'].apply(gender_to_numeric)

# check the data
print(diabetes_df.head())
```

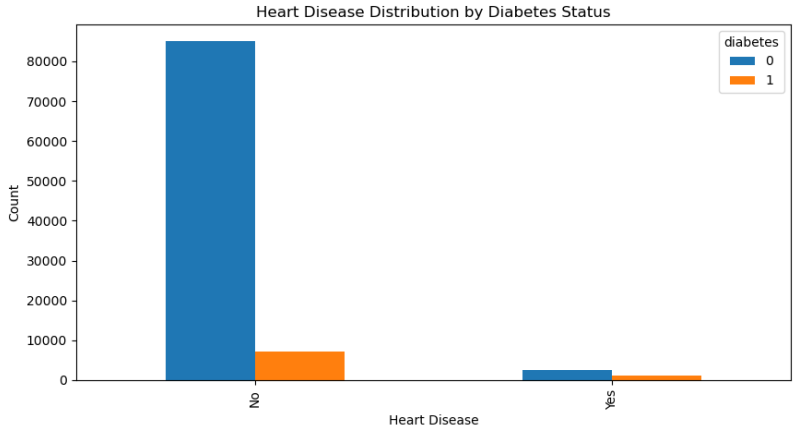
```python
# define custom function for the smoking history conversion
def smoking_history_to_numeric(smoking):
    if smoking == "current":
        return 0
    elif smoking == "ever":
        return 1
    elif smoking == "former":
        return 2
    elif smoking == "never":
        return 3
    elif smoking == "not current":
        return 4
    elif smoking == "No Info":
        return 5

# use apply function to transform the categorical column into a numeric column
diabetes_df['smoking_history'] = diabetes_df['smoking_history'].apply(smoking_history_to_numeric)

# check the data
print(diabetes_df.head())
```

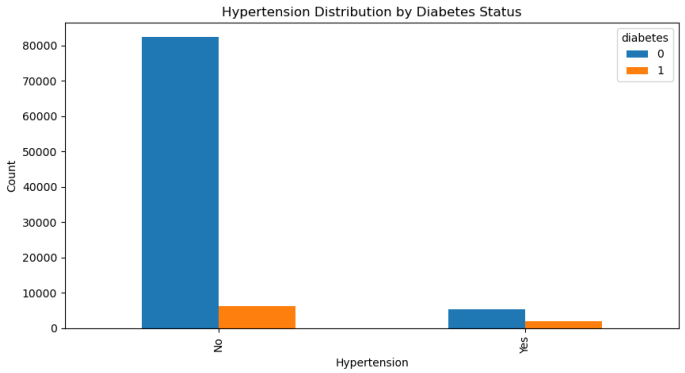
Converted categorical variables into numerical values so that machine-learning algorithms could process them. **gender** was converted to 0/1, while **smoking_history** was mapped to numerical categories from 0 to 5.

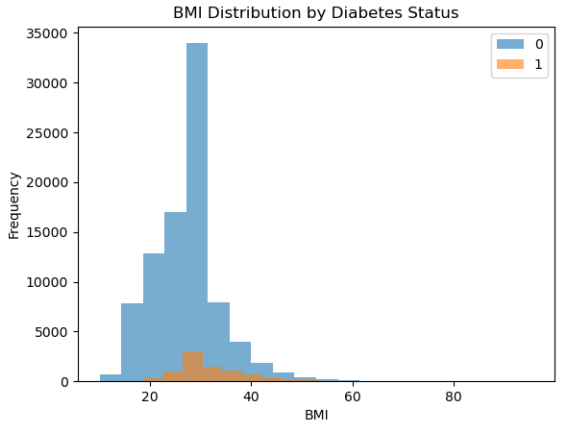
EXPLORATORY DATA ANALYSIS (EDA)

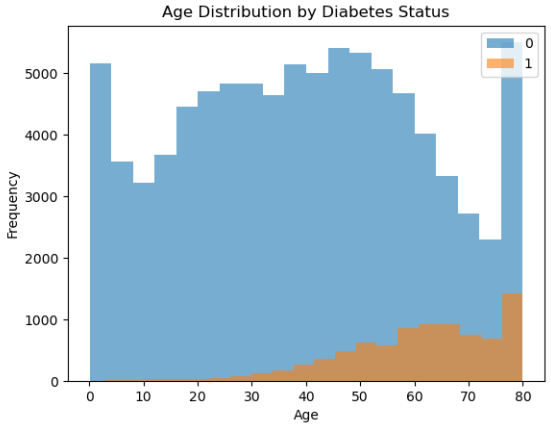


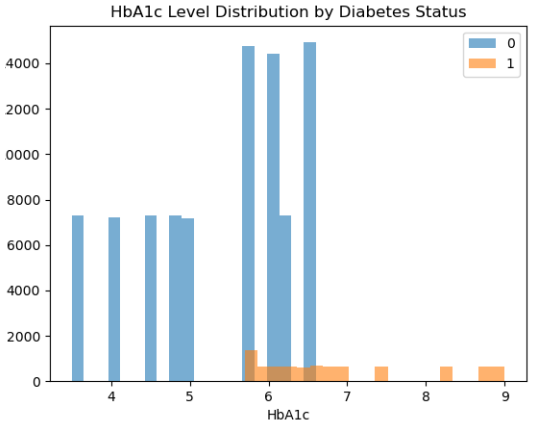












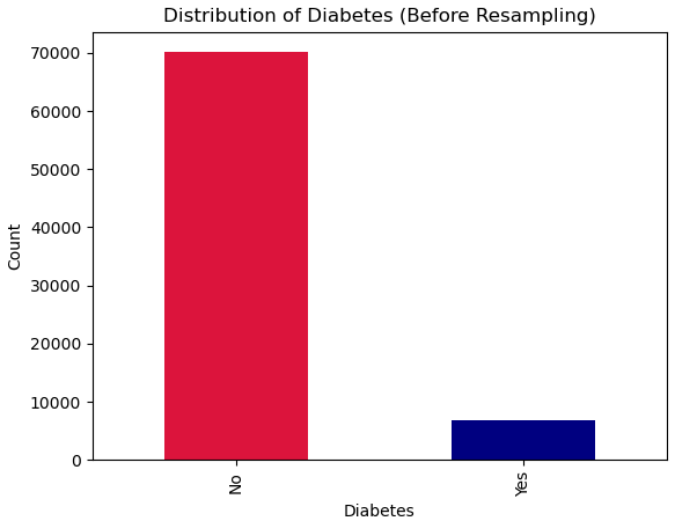
TRAIN-TEST SPLIT

```
# split the data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(diabetes_df.drop(columns = ['diabetes'], axis = 1),
                                                    diabetes_df['diabetes'],
                                                    test_size = 0.20,
                                                    shuffle = True,
                                                    stratify = diabetes_df['diabetes'],
                                                    random_state = 42)
```

Divided the dataset into 80% training data and 20% testing data. Stratified sampling was used to maintain a similar diabetes class distribution in both datasets.

CLASS DISTRIBUTION ANALYSIS

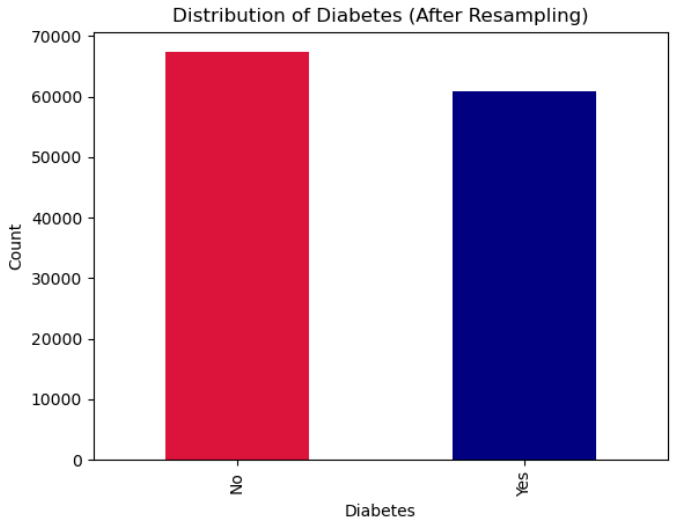
```
# before applying resampling
y_train.value_counts().plot(kind = "bar", color = ['crimson', 'navy'])
plt.title("Distribution of Diabetes (Before Resampling)")
plt.xlabel("Diabetes")
plt.ylabel("Count")
plt.xticks(ticks = [0, 1], labels = ["No", "Yes"])
plt.savefig("before_resampling_diabetes.png")
plt.close()
```



Examined the distribution of diabetes classes in the training data before resampling to identify class imbalanced.

CLASS IMBALANCE HANDLING

```
# apply SMOTE-Tomek
smt = SMOTEENN(random_state = 42)
X_train_res, y_train_res = smt.fit_resample(X_train, y_train)

# after applying resampling
y_train_res.value_counts().plot(kind = "bar", color = ['crimson', 'navy'])
plt.title("Distribution of Diabetes (After Resampling)")
plt.xlabel("Diabetes")
plt.ylabel("Count")
plt.xticks(ticks = [0, 1], labels = ["No", "Yes"])
plt.savefig("after_resampling_diabetes.png")
plt.close()
```



Applied SMOTEENN to the training data. This combines synthetic oversampling with cleaning of potentially noisy or ambiguous observations to produce a more balanced training dataset.

FEATURE STANDARDIZATION

```
scaler = StandardScaler()
X_train_res[['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']] = scaler.fit_transform(X_train_res[['age', 'bmi', 'HbA1c_level', 'blood_glucose_le
X_test[['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']] = scaler.transform(X_test[['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']])
```

Applied **StandardScaler** to numerical variables – age, BMI, HbA1c level, and blood glucose level. The scaler was fitted only on the resampled training data and then applied to the test data.

MODEL TRAINING & HYPERPARAMETER TUNING

```
# initialize stratified k-fold
cv = StratifiedKFold(n_splits = 5, shuffle = True, random_state = 42)

# random forest classifier
param_grid_rfc = {'n_estimators': [10, 20, 30, 40, 50],
                  'max_depth': [3, 5, 7]}
gs_rfc = GridSearchCV(estimator = RandomForestClassifier(random_state = 42),
                      param_grid = param_grid_rfc,
                      cv = cv,
                      scoring = "accuracy",
                      n_jobs = 1,
                      return_train_score = True,
                      verbose = 3)
gs_rfc.fit(X_train_res, y_train_res)
```

```
# logistic regression
param_grid_lg = {'C': [0.1, 0.01, 1]}
gs_lg = GridSearchCV(estimator = LogisticRegression(random_state = 42),
                     param_grid = param_grid_lg,
                     cv = cv,
                     scoring = "accuracy",
                     n_jobs = 1,
                     return_train_score = True,
                     verbose = 3)
gs_lg.fit(X_train_res, y_train_res)

# decision trees classifier
param_grid_dtc = { 'max_depth': [3, 5, 7]}
gs_dtc = GridSearchCV(estimator = DecisionTreeClassifier(random_state = 42),
                      param_grid = param_grid_dtc,
                      cv = cv,
                      scoring = "accuracy",
                      n_jobs = 1,
                      return_train_score = True,
                      verbose = 3)
gs_dtc.fit(X_train_res, y_train_res)
```

Trained three classification models: **Random Forest, Logistic Regression, and Decision Tree. GridSearchCV** with **5-fold Stratified Cross-Validation** was used to test different hyperparameter combinations and select the best-performing configuration based on accuracy.

MODEL PREDICTION

```
# perform prediction on train set and test set
y_pred_rfc_train = gs_rfc.predict(X_train_res)
y_pred_rfc_test = gs_rfc.predict(X_test)

y_pred_lg_train = gs_lg.predict(X_train_res)
y_pred_lg_test = gs_lg.predict(X_test)

y_pred_dtc_train = gs_dtc.predict(X_train_res)
y_pred_dtc_test = gs_dtc.predict(X_test)
```

Used each optimized model to generate predictions for both the training data and the unseen test data. This allowed comparison of model performance on data used for training versus unseen data.

MODEL EVALUATION

```
# classification report for random forest classifier
print("Classification Report for Random Forest Classifier")
print("============================================================")
print("TRAIN: ")
print(classification_report(y_train_res, y_pred_rfc_train))
print("TEST: ")
print(classification_report(y_test, y_pred_rfc_test))

# classification report for logistic regression
print("Classification Report for Logistic Regression")
print("============================================================")
print("TRAIN: ")
print(classification_report(y_train_res, y_pred_lg_train))
print("TEST: ")
print(classification_report(y_test, y_pred_lg_test))

# classification report for decision tree classifier
print("Classification Report for Random Forest Classifier")
print("============================================================")
print("TRAIN: ")
print(classification_report(y_train_res, y_pred_dtc_train))
print("TEST: ")
print(classification_report(y_test, y_pred_dtc_test))
```

Evaluated the three classifiers using classification reports, including precision, recall, F1-score, and accuracy. Both training and test performance were examined to assess predictive performance and possible overfitting.

MODEL EVALUATION

Classification Report for Random Forest Classifier

TRAIN:

	precision	recall	f1-score	support
0	0.97	0.93	0.95	60979
1	0.94	0.97	0.96	67377
accuracy			0.95	128356
macro avg	0.95	0.95	0.95	128356
weighted avg	0.95	0.95	0.95	128356

TEST:

	precision	recall	f1-score	support
0	0.99	0.86	0.92	17534
1	0.39	0.92	0.55	1696
accuracy			0.87	19230
macro avg	0.69	0.89	0.74	19230
weighted avg	0.94	0.87	0.89	19230

Classification Report for Logistic Regression

TRAIN:

	precision	recall	f1-score	support
0	0.93	0.92	0.92	60979
1	0.93	0.94	0.93	67377
accuracy			0.93	128356
macro avg	0.93	0.93	0.93	128356
weighted avg	0.93	0.93	0.93	128356

TEST:

	precision	recall	f1-score	support
0	0.99	0.86	0.92	17534
1	0.38	0.90	0.53	1696
accuracy			0.86	19230
macro avg	0.68	0.88	0.72	19230
weighted avg	0.93	0.86	0.88	19230

Classification Report for Decision Tree Classifier

TRAIN:

	precision	recall	f1-score	support
0	0.96	0.93	0.94	60979
1	0.94	0.96	0.95	67377
accuracy			0.95	128356
macro avg	0.95	0.95	0.95	128356
weighted avg	0.95	0.95	0.95	128356

TEST:

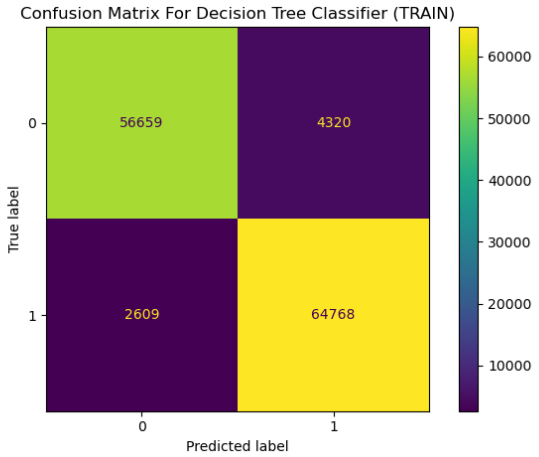
	precision	recall	f1-score	support
0	0.99	0.86	0.92	17534
1	0.39	0.92	0.55	1696
accuracy			0.87	19230
macro avg	0.69	0.89	0.73	19230
weighted avg	0.94	0.87	0.89	19230

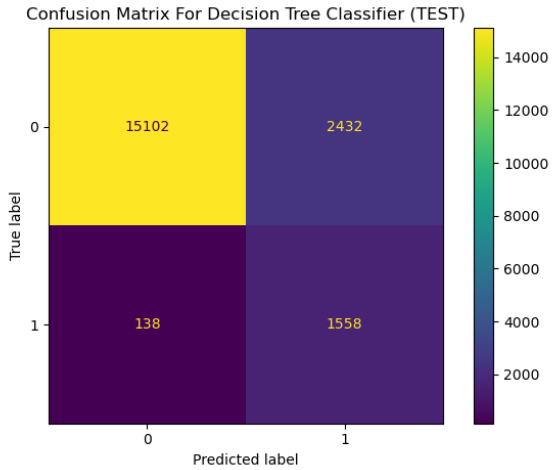
CONFUSION MATRIX ANALYSIS

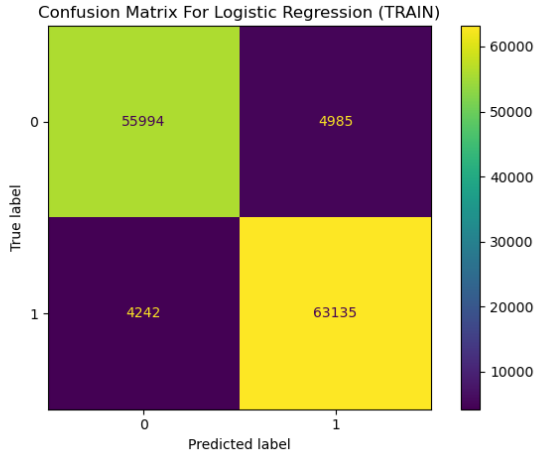
```
# confusion matrix for random forest classifier
disp_rfc_train = ConfusionMatrixDisplay.from_predictions(y_train_res, y_pred_rfc_train)
plt.title("Confusion Matrix For Random Forest Classifier (TRAIN)")
plt.tight_layout()
plt.savefig("confusion_matrix_rfc_train.png")
plt.close()

disp_rfc_test = ConfusionMatrixDisplay.from_predictions(y_test, y_pred_rfc_test)
plt.title("Confusion Matrix For Random Forest Classifier (TEST)")
plt.tight_layout()
plt.savefig("confusion_matrix_rfc_test.png")
plt.close()
```

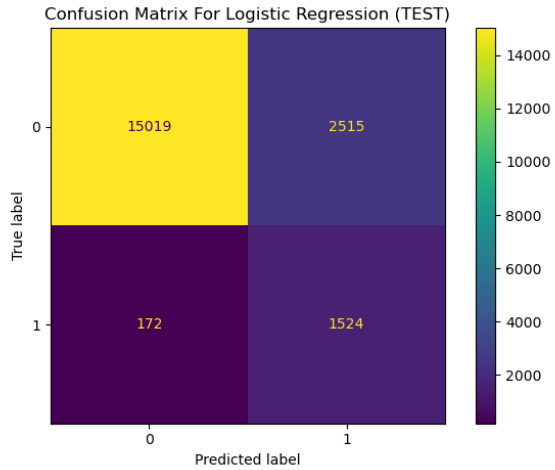
Generated confusion matrices for Random Forest, Logistic Regression, and Decision Tree on both training and test datasets to examine correct and incorrect diabetes classifications.

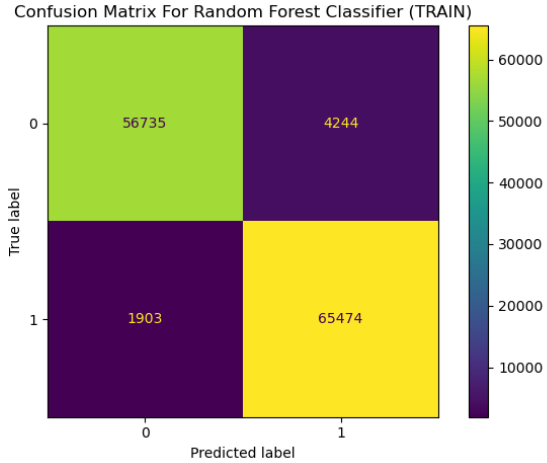
CONFUSION MATRIX ANALYSIS

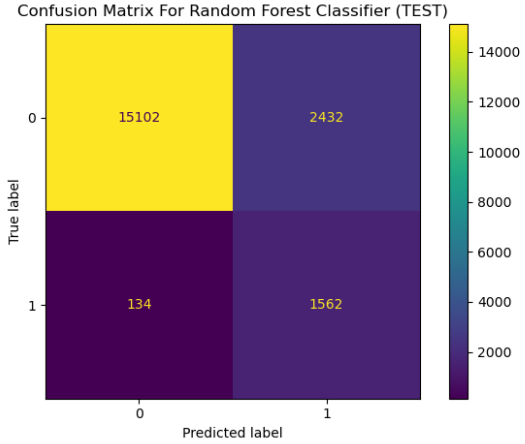




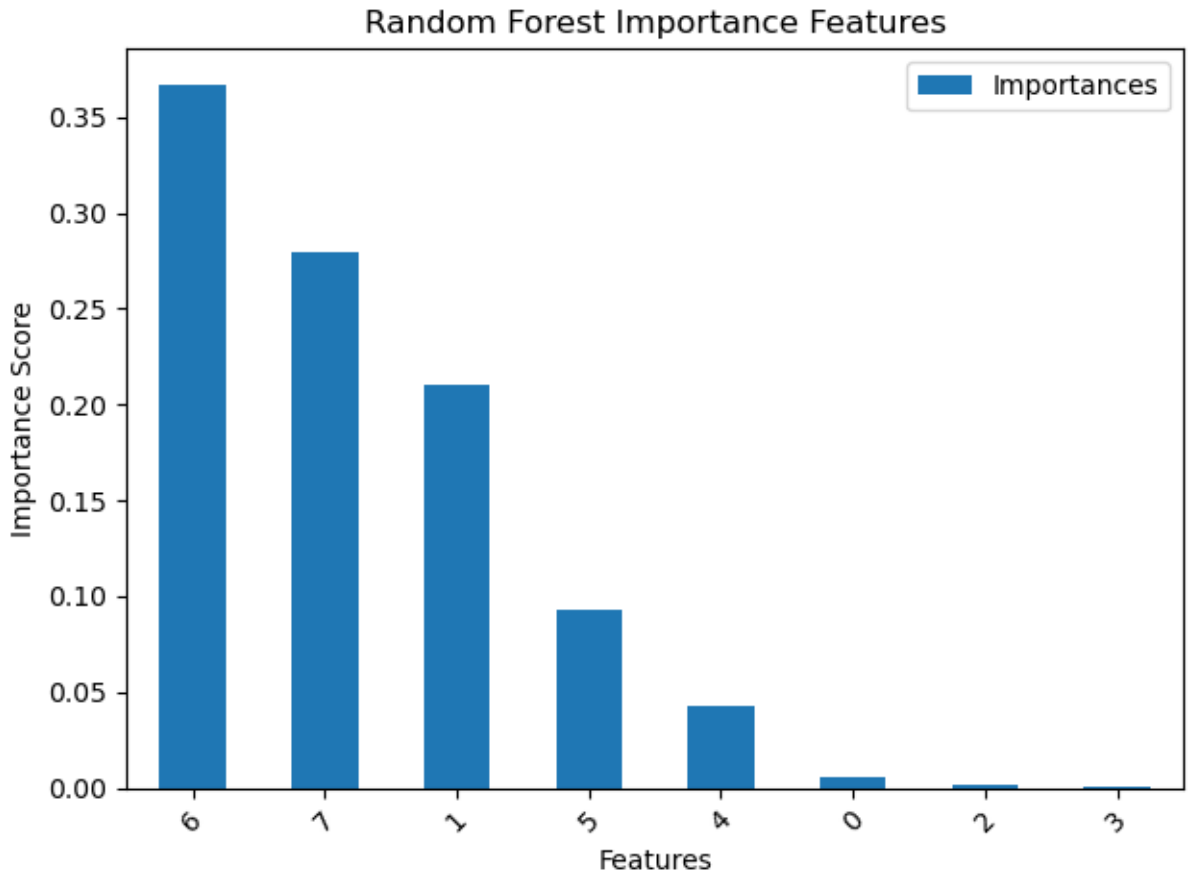








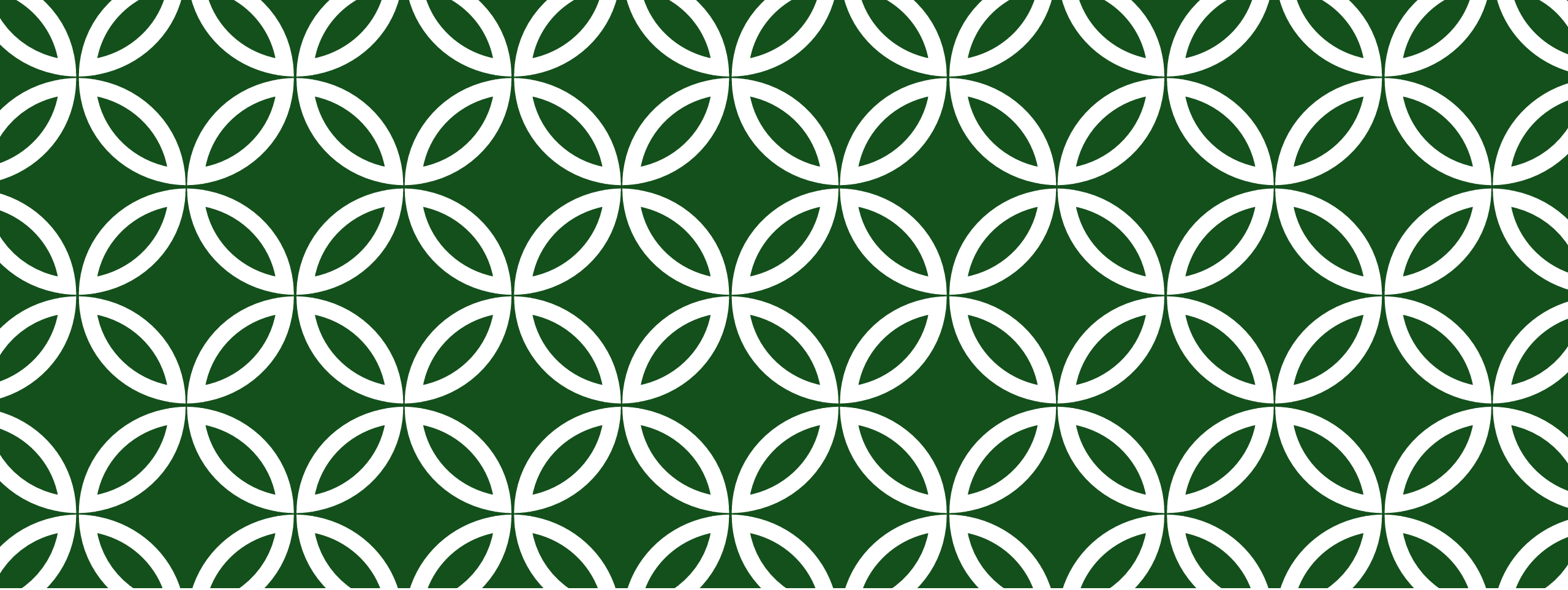
FEATURE IMPORTANCES OF BEST MODEL



MODEL DEPLOYMENT PREPARATION

```
# save the models and scalers
joblib.dump(gs_rfc.best_estimator_, "random_forest_model.joblib")
joblib.dump(scaler, "scaler.joblib")
```

Saved the best Random Forest estimator from the grid search as random_forest_model.joblib and saved the fitted scaler as scaler.joblib. These files can later be loaded by an application/ API to make predictions on new data.



DEMO WORKING IMPLEMENTATION